RV College of Engineering®

Go, Change the World

Academic Year 2024-25 (ODD Semester)

USN	1	R	V							

Department of Artificial Intelligence and Machine Learning

ARTIFICIAL NEURAL NETWORKS AND DEEP LEARNING

Course Code	:	**AI253IA**	**Date**	:	**28/1/2025**
Semester	:	**V Sem**	**Time**	:	**9:30am to 11:30am**
Max Marks	:	**10 (Q) + 50(CIE)**	**Duration**	:	**30 + 90=120 min**

Improvement Quiz

Note: Answer all the Questions

Q. No		Questions	M	BT	CO
1	a)	**Representation**: Modeling states focuses on estimating values based on the environment's states, while modeling state-action pairs estimates values for specific actions taken in given states. **Policy Dependence**: State-value models (V-values) depend on the overall policy, whereas state-action models (Q-values) directly evaluate action choices, making them more suitable for policy improvement.	2	2	1
	b)	**Exploration-Exploitation Tradeoff**: Balancing between exploring new actions to discover better rewards and exploiting known actions to maximize immediate gains is challenging. **Sample Inefficiency**: RL often requires a large number of interactions with the environment to learn effective policies, which can be computationally expensive and time-consuming.	2	1	1
	c)	Bonus(s,a)=c·N(s,a)lnt	2	3	1
	d)	expected cumulative reward	2	2	2
	e)	In **Reinforcement Learning (RL)**, **bootstrapping** refers to the process of **updating a value estimate using another estimated value instead of waiting for the final reward**. This allows learning to happen efficiently by leveraging existing estimates rather than relying solely on complete episode outcomes.	2	2	1

Improvement Test

Note: Answer all the Questions

Q. No		Questions	M	BT	CO
2	a)	**Update Rule** • **SARSA**: Uses the action selected by the current policy for updating. Q(s,a)←Q(s,a)+α[R+γQ(s′,a′)−Q(s,a)]Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma Q(s', a') - Q(s, a) \right]Q(s,a)←Q(s,a)+α[R+γQ(s′,a′)−Q(s,a)] • **Q-Learning**: Uses the maximum possible future Q-value, making it independent of the policy. Q(s,a)←Q(s,a)+α[R+γmax a′Q(s′,a′)−Q(s,a)]Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') - Q(s, a)	5	1	2

		\right]Q(s,a)←Q(s,a)+α[R+γa′maxQ(s′,a′)−Q(s,a)] **Exploration vs. Exploitation** • **SARSA**: Follows the current policy, making it more **on-policy** (exploration-aware). • **Q-Learning**: Updates based on the best possible action, making it **off-policy** (more greedy). **Behavior in Stochastic Environments** • **SARSA**: Safer in **stochastic** environments since it accounts for the policy's actual actions. • **Q-Learning**: Can be more aggressive and may perform poorly in highly stochastic settings. **Convergence and Learning Stability** • **SARSA**: More stable because it learns based on actions taken under the current policy. • **Q-Learning**: Can converge faster but may be unstable if the environment is dynamic. **Application Suitability** • **SARSA**: Better suited for **safe learning**, such as robot control or navigation in uncertain environments. • **Q-Learning**: More suitable for **maximizing rewards**, such as game-playing and optimal decision-making in deterministic settings. 1M -Each			
3	**b)**	In RL, a **function approximator** is used to estimate value functions like Q(s,a)Q(s, a)Q(s,a) when the state and action spaces are too large to store exact values in a table. A **neural network (NN)** can be used to approximate these functions efficiently. Example: Deep Q-Network (DQN) for Playing Atari Games One famous example of using neural networks as function approximators in RL is **Deep Q-Network (DQN)**, which was developed by DeepMind to play **Atari games** at human-level performance. *How It Works:* **Input Representation:** ○ The **state** sss is represented as a stack of recent frames (images) from the game. ○ These images are processed as pixel values (e.g., 84×8484 \times 8484×84 grayscale images). **Neural Network as a Q-Function Approximator:** ○ Instead of maintaining a **Q-table**, a **Convolutional Neural Network (CNN)** is used. ○ The NN takes in the state sss and outputs estimated **Q-values** for all possible actions aaa.	5	2	3

- The highest Q-value action is chosen for execution.

Training with Experience Replay:

- Transitions (s,a,r,s')(s, a, r, s')(s,a,r,s') are stored in a **replay buffer**.
- During training, random samples from the buffer are used to break correlation between consecutive game frames.
- The loss function used is the **Mean Squared Error (MSE)** of the Bellman equation: Loss=(R+γmax a'Q(s',a')−Q(s,a))2\text{Loss} = \left(R + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)^2Loss=(R+γa'maxQ(s',a')−Q(s,a))2

Target Network for Stability:

- A separate **target network** is used to stabilize training by updating it periodically instead of every step.

1M-Each

a) 5 2 2

Feature	GANs (Generative Adversarial Networks)	VAEs (Variational Autoencoders)
Architecture	Consists of a **Generator** and a **Discriminator** in an adversarial setup.	Based on an **encoder-decoder** architecture with a latent space following a probability distribution.
Training Process	Uses a **min-max optimization** where the generator tries to fool the discriminator while the discriminator learns to distinguish real from fake samples.	Uses **variational inference** by maximizing the evidence lower bound (ELBO) to approximate a probabilistic latent space.
Loss Function	Uses **adversarial loss** (binary cross-entropy) to optimize both networks.	Uses **reconstruction loss** (e.g., MSE) and **KL divergence** to regularize the latent space.
Quality of Generated Samples	Produces **sharp and realistic** samples but may suffer from **mode collapse** (limited diversity).	Produces **diverse but blurry** samples due to enforcing smooth latent space regularization.
Latent Space Structure	Unstructured, meaning later ctors do not necessarily correspond to meaningful	Structured latent space, enabling controlled generation and smooth

b) 5 2 2

Attention Mechanism in Deep Learning

The **Attention Mechanism** is a technique that allows deep learning models to **focus on the most relevant parts of input data** when making predictions. It dynamically assigns different importance (weights) to different elements in the input, improving performance in tasks like **natural language processing (NLP), computer vision, and speech recognition**

Role of Attention in Deep Learning Models

Enhancing Sequence Processing

- Traditional sequence models (like RNNs) suffer from the **vanishing gradient problem**, making it difficult to handle long-term dependencies.
- Attention helps the model selectively focus on key parts of a sequence, improving performance in tasks like **machine translation** and **text summarization**.

Improving Computational Efficiency

- Unlike RNNs, which process inputs sequentially, attention-based models (like **Transformers**) can process all input tokens **in parallel**, significantly reducing training time.

Better Context Understanding

- In NLP, attention mechanisms allow models to **weigh different words differently** depending on their relevance, improving understanding in tasks

		like **question answering** and **sentiment analysis**. **Applications Beyond NLP** ○ In **computer vision**, **self-attention mechanisms** (e.g., Vision Transformers) help models focus on important image regions without relying on fixed-size convolutional filters. ○ In **speech recognition**, attention helps models focus on important sound patterns to improve transcription accuracy. 1m-EACH			
4	**a)**	Monte Carlo Tree Search (**MCTS**) is a **heuristic search algorithm** used in decision-making for games like **Go, Chess, and Shogi**. It efficiently determines the best move by **simulating possible future game states** and **balancing exploration and exploitation** during the search. In **Go**, MCTS helps AI (e.g., AlphaGo) navigate the vast number of possible moves by intelligently exploring the game tree instead of brute-force searching all possible sequences. Key Steps in the MCTS Algorithm MCTS iteratively builds a search tree by repeating the following **four main steps** **Selection** ○ The algorithm starts at the root node (current game state) and **selects a promising child node** based on a balance of **exploration and exploitation**. ○ Selection is often guided by the **Upper Confidence Bound for Trees (UCT)** formula: UCT=WN+cln TNUCT = \frac{W}{N} + c \sqrt{\frac{\ln T}{N}}UCT=NW+cNlnT	5	2	1
	b)	Conditional Generative Adversarial Networks (cGANs) **Conditional GANs (cGANs)** are an extension of the basic **Generative Adversarial Networks (GANs)** that allow for more **controlled generation** of data by conditioning both the **generator** and the **discriminator** on additional information, such as class labels or input images. This enables the generator to produce samples based on specific conditions, making cGANs useful in tasks like **image generation with specific attributes** **image-to-image translation**, and **text-to-image synthesis** How cGANs Extend the Basic GAN Architecture In a traditional GAN, the generator GGG learns to generate data that resembles the true data distribution, while the discriminator DDD tries to differentiate between real and fake data. The model training follows a **min-max game** min Gmax DEx~pdata(x)[log D(x)]+Ez~pz(z)[log (1−D(G(z)))]\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}(x)}[\log D(x)] + \mathbb{E}_{z \sim p_{\text{z}}(z)}[\log(1 - D(G(z)))]GminDmaxEx~pdata(x)[logD(x)]+Ez~pz(z)[log(1−D(G(z)))] In a **cGAN**, we introduce a **condition** (such as a label or an image) that guides both the generator and the discriminator, which helps create more controlled and meaningful outputs.	5	3	3

Key Changes in cGANs

Conditional Generator

- The generator GGG is conditioned on some additional information yyy. This could be:
  - **Class labels**: Generating images of a specific class, e.g., cats, dogs, etc.
  - **Input images**: For image-to-image translation tasks (e.g., turning sketches into realistic images).
  - **Text descriptions**: Generating images based on textual descriptions (e.g., generating an image of "a red apple").
- The generator now takes a combination of noise vector zzz and the condition yyy as input: G(z,y)G(z, y)G(z,y) This ensures the generated sample is influenced by the condition yyy.

Conditional Discriminator

- Similarly, the discriminator DDD is also conditioned on the same information yyy along with the input image (real or fake). It learns to classify whether the image is real or fake, given the condition.
- The discriminator evaluates whether the image xxx matches the condition yyy in addition to determining whether it is real or fake: D(x,y)D(x, y)D(x,y)

Training Objective

The objective in cGANs is similar to the original GAN but incorporates the condition yyy in both the generator and discriminator:

min Gmax DEx~pdata(x|y)[log D(x,y)]+Ez~pz(z)[log (1−D(G(z,y),y))]\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}(x | y)}[\log D(x, y)] + \mathbb{E}_{z \sim p_{\text{z}}(z)}[\log(1 - D(G(z, y), y))]GminDmaxEx~pdata(x|y)[logD(x,y)]+Ez~pz(z)[log(1−D(G(z,y),y))]

Here, the discriminator tries to distinguish between real and generated images, conditioned on yyy, while the generator tries to produce images that fool the discriminator into thinking they are real.

5		Generative Adversarial Networks (GANs) Architecture and Training Process	10	2	3

Generative Adversarial Networks (GANs) are a class of machine learning models designed for **generative tasks**, such as image, video, or audio generation. They consist of two primary components: a **Generator** and a **Discriminator**, which work together in a **min-max game** to produce realistic data.

1. GANs Architecture

The architecture of GANs involves two neural networks:

Generator (G)

- **Purpose**: The generator creates fake data that mimics real data, such as generating images that resemble real-world images.
- **Input**: The generator takes a random vector (also called noise or latent vector), typically sampled from a simple distribution like a Gaussian (normal distribution).
- **Output**: The generator produces data that is intended to appear as if

it belongs to the distribution of real data.

- **Mathematical formulation**

  G(z)=x^G(z) = \hat{x}G(z)=x^

  where zzz is the noise vector, and x^\hat{x}x^ is the generated sample.

Discriminator (D)

- **Purpose**: The discriminator's job is to classify whether a given input sample is real (from the true data distribution) or fake (from the generator).
- **Input**: The discriminator takes data samples as input, either from the real dataset or the generated fake data.
- **Output**: The discriminator outputs a probability value (between 0 and 1), where 1 means the input is real and 0 means the input is fake.
- **Mathematical formulation**

  D(x)=probability that x is realD(x) = \text{probability that } x \text{ is real}D(x)=probability that x is real

2. GANs Training Process (Min-Max Game)

The training process of GANs can be viewed as a **two-player game** between the generator and the discriminator, where the **generator tries to generate fake data** to fool the discriminator, and the **discriminator tries to distinguish real data from fake data**

The training follows an iterative process, where both networks are updated alternatively:

Step-by-Step Training:

Discriminator Training

- **Objective**: Train the discriminator to correctly classify real and fake samples.
- During each training iteration:

  Sample a batch of **real data** (e.g., real images from the dataset).
  Sample a batch of **fake data** generated by the current generator.
  Update the discriminator to maximize the probability of assigning the correct label to real and fake samples (i.e., 1 for real and 0 for fake).

  - **Loss function for discriminator**:
    LD=−Ex~pdata(x)[log D(x)]−Ez~pz(z)[log (1−D(G(z)))]L_D = -\mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] - \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]LD=−Ex~pdata(x)[logD(x)]−Ez~pz(z)[log(1−D(G(z)))] where D(x)D(x)D(x) is the discriminator's output for real data xxx, and G(z)G(z)G(z) is the fake data generated from noise zzz.

Generator Training

- **Objective**: Train the generator to produce fake data that **fools the**

discriminator into classifying it as real.

- The generator is updated to minimize the discriminator's ability to distinguish real from fake data.
  - The generator's loss is based on the **discriminator's output** for fake data. The generator aims to **maximize** D(G(z))D(G(z))D(G(z)), so the output of the discriminator is close to 1 (i.e., the discriminator thinks the fake data is real).
  - **Loss function for generator**

    LG=−Ez∼pz(z)[log D(G(z))]L_G = -\mathbb{E}_{z \sim p_z(z)} [\log D(G(z))]LG=−Ez∼pz(z)[logD(G(z))]

    where D(G(z))D(G(z))D(G(z)) is the probability that the discriminator assigns to the fake data generated by the generator.

3. Min-Max Optimization

The **objective of GANs** is to train the **generator and discriminator in tandem** using the **min-max game**

- The **discriminator** tries to **maximize** its ability to distinguish real and fake data.
- The **generator** tries to **minimize** its ability to fool the discriminator, which is equivalent to maximizing the discriminator's classification of fake data as real.

The overall objective function is:

min Gmax DEx∼pdata(x)[log D(x)]+Ez∼pz(z)[log (1−D(G(z)))]\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]GminDmaxEx∼pdata(x)[logD(x)]+Ez∼pz(z)[log(1−D(G(z)))]

The generator aims to minimize log (1−D(G(z)))\log(1 - D(G(z)))log(1−D(G(z))) (i.e., fooling the discriminator into believing its fake samples are real), while the discriminator aims to maximize its ability to distinguish between real and fake data.

4. Balancing the Generator and Discriminator

- **Training Stability**: GANs are notoriously difficult to train because the generator and discriminator can easily become imbalanced. If one network (e.g., the discriminator) becomes too strong, the other (e.g., the generator) may fail to improve.
  - **Tips for improving stability**:
    - Use techniques like **feature matching**, **historical averaging**, and **gradient penalty** to stabilize training.
    - Use **better initialization** and **regularization** techniques.
- **Convergence**: GANs theoretically converge when the generator produces data that is indistinguishable from real data, meaning the discriminator can no longer distinguish between real and fake samples.

6		The **Multi-Armed Bandit (MAB)** problem is a classic reinforcement learning problem that involves a **fixed set of actions** (called arms) with unknown rewards,	10	3	2

and the objective is to maximize the cumulative reward over time. The name comes from the metaphor of a gambler at a row of slot machines (the "arms"), each with a different, unknown probability distribution for the payout. The challenge is deciding which arm to pull (action) at each step to maximize the overall reward.

Why is MAB Considered Statless?

In the **Multi-Armed Bandit** problem, the key assumption is that each action's **reward is independent of the previous actions**. The rewards are **not dependent on a sequence of past actions or states**; hence, there is no notion of state transitions. The problem is **stateless** because:

- **No state information**: Each decision is made solely based on the current arm's reward distribution, without considering the history of previous arms chosen or the previous outcomes.
- The **environment is independent**: The reward for each action is drawn from a fixed, unknown distribution and does not depend on the history of the environment.

This is different from problems like **Markov Decision Processes (MDPs)** where actions influence future states, and rewards depend on both the current state and the chosen action.

Solving the MAB Problem with Naïve Bayes and ε-Greedy Algorithms

1. Naïve Bayes Algorithm

Naïve Bayes is a probabilistic classification algorithm that applies **Bayes' theorem** under the assumption that the features are independent. In the context of the MAB problem, Naïve Bayes can be applied to estimate the **probability distribution of rewards** for each arm.

- **How it works**
  - For each arm, we maintain a probability distribution (e.g., Gaussian, Bernoulli) representing the estimated reward distribution.
  - Naïve Bayes updates the distribution based on the observed rewards each time an arm is chosen.
  - We assume that the reward for each arm follows a **specific probabilistic model**, and we update this model as more data (rewards) is collected.
  - The algorithm chooses the arm that maximizes the **expected reward**, which can be estimated by the probability of obtaining a high reward based on the current model.
- **Steps**

1. Initialize a probabilistic model for each arm (mean and variance if assuming Gaussian distribution).
2. Select an arm based on the current beliefs (e.g., the one with the highest expected reward).
3. Observe the reward and update the model using Bayes' theorem.
4. Repeat the process.

- **Advantages**
  - Can model complex reward distributions.
  - Gradually refines estimates with more data.
- **Drawbacks**

- **Assumption of independence**: Naïve Bayes assumes that features (rewards for arms) are independent, which may not always be true in practice.
- Requires **prior knowledge of distributions**: If the true reward distribution is unknown or different from the assumed model, the algorithm might perform poorly.
- **Computational complexity**: As the number of arms grows, updating and maintaining probabilistic models for each arm can become computationally expensive.

2. ε-Greedy Algorithm

The **ε-Greedy** algorithm is a simple, popular approach for balancing **exploration** (trying new arms) and **exploitation** (choosing the best-performing arm).

- **How it works**
  - The algorithm maintains an estimate of the **average reward** for each arm.
  - With probability **ε**, it selects an arm at random (exploration).
  - With probability **1 - ε**, it selects the arm that has the **highest average reward** (exploitation).
  - Over time, as more rewards are observed, the estimate of each arm's average reward is updated.
- **Steps**

1. Initialize estimates of the rewards for all arms (usually to 0 or some neutral value).
2. At each step, choose an arm:
   - With probability ε, select an arm randomly (explore).
   - With probability 1-ε, select the arm with the highest average reward (exploit).
3. After selecting an arm, observe the reward and update the estimate of that arm's average reward.
4. Repeat the process.

- **Advantages**
  - **Simple and effective**: Easy to implement and works well in many cases.
  - Balances exploration and exploitation with an adjustable **ε** value.
  - Does not assume any knowledge about the reward distribution.
- **Drawbacks**
  - **Fixed exploration rate**: The ε value is typically constant. This means that exploration doesn't decrease over time, even as more information is gained, potentially leading to wasted exploration.
  - **Slow convergence**: The algorithm might not converge quickly to the optimal arm, especially with a high exploration rate (large ε).
  - **Suboptimal performance**: If ε is too large, the algorithm spends too much time exploring, leading to suboptimal performance. Conversely, if ε is too small, the algorithm may over-exploit and miss better arms.

M-Marks, BT-Blooms Taxonomy Levels, CO-Course Outcomes

Marks Distribution	Particulars	CO1	CO2	CO3	CO4	L1	L2	L3	L4	L5	L6
	Max Marks CIE & Quiz	13	27	20	--	--	--	--	--	--	--

Course Outcomes

CO1 :	Describe basic concepts of neural networks, its applications and various learning models
CO2 :	Analyze different network architectures, learning tasks, CNN, and deep learning models
CO3 :	Investigate and apply neural networks model and learning techniques to solve problems related to society and industry.
CO4 :	Demonstrate a prototype application developed using any NN tools and APIs.
CO5	Appraise the knowledge of neural networks and deep learning as an individual/as an team member.